

*Applicability of this chapter to configurations:*

This chapter is applicable to all CV32A6 configurations.

| Configuration | Implementation        |
|---------------|-----------------------|
| CV32A60AX     | Implemented extension |
| CV32A60X      | Implemented extension |
| CV64A6_MMU    | Implemented extension |

**Note:** CV64A6 implements RV64I that includes additional instructions.

## RV32I Base Integer Instruction Set

This section describes the RV32I base integer instruction set.

### Integer Register-Immediate Instructions

- **ADDI:** Add Immediate

**Format:** `addi rd, rs1, imm[11:0]`

**Description:** add sign-extended 12-bit immediate to register rs1, and store the result in register rd.

**Pseudocode:**  $x[rd] = x[rs1] + \text{sext}(\text{imm}[11:0])$

**Invalid values:** NONE

**Exception raised:** NONE

- **ANDI:** AND Immediate

**Format:** andi rd, rs1, imm[11:0]

**Description:** perform bitwise AND on register rs1 and the sign-extended 12-bit immediate and place the result in rd.

**Pseudocode:**  $x[rd] = x[rs1] \& \text{sext}(\text{imm}[11:0])$

**Invalid values:** NONE

**Exception raised:** NONE

- **ORI:** OR Immediate

**Format:** ori rd, rs1, imm[11:0]

**Description:** perform bitwise OR on register rs1 and the sign-extended 12-bit immediate and place the result in rd.

**Pseudocode:**  $x[rd] = x[rs1] | \text{sext}(\text{imm}[11:0])$

**Invalid values:** NONE

**Exception raised:** NONE

- **XORI:** XOR Immediate

**Format:** xori rd, rs1, imm[11:0]

**Description:** perform bitwise XOR on register rs1 and the sign-extended 12-bit immediate and place the result in rd.

**Pseudocode:**  $x[rd] = x[rs1] \wedge \text{sext}(\text{imm}[11:0])$

**Invalid values:** NONE

**Exception raised:** NONE

- **SLTI:** Set Less Than Immediate

**Format:** slti rd, rs1, imm[11:0]

**Description:** set register rd to 1 if register rs1 is less than the sign extended immediate when both are treated as signed numbers, else 0 is written to rd.

**Pseudocode:** if  $(x[rs1] < sext(imm[11:0]))$   $x[rd] = 1$  else  $x[rd] = 0$

**Invalid values:** NONE

**Exception raised:** NONE

- **SLTIU:** Set Less Than Immediate Unsigned

**Format:** sltiu rd, rs1, imm[11:0]

**Description:** set register rd to 1 if register rs1 is less than the sign extended immediate when both are treated as unsigned numbers, else 0 is written to rd.

**Pseudocode:** if  $(x[rs1] <_u sext(imm[11:0]))$   $x[rd] = 1$  else  $x[rd] = 0$

**Invalid values:** NONE

**Exception raised:** NONE

- **SLLI:** Shift Left Logic Immediate

**Format:** slli rd, rs1, imm[4:0]

**Description:** logical left shift (zeros are shifted into the lower bits).

**Pseudocode:**  $x[rd] = x[rs1] \ll imm[4:0]$

**Invalid values:** NONE

**Exception raised:** NONE

- **SRLI:** Shift Right Logic Immediate

**Format:** srli rd, rs1, imm[4:0]

**Description:** logical right shift (zeros are shifted into the upper bits).

**Pseudocode:**  $x[rd] = x[rs1] \gg imm[4:0]$

**Invalid values:** NONE

**Exception raised:** NONE

- **SRAI:** Shift Right Arithmetic Immediate



**Format:** srai rd, rs1, imm[4:0]

**Description:** arithmetic right shift (the original sign bit is copied into the vacated upper bits).

**Pseudocode:**  $x[rd] = x[rs1] \gg s\ imm[4:0]$

**Invalid values:** NONE

**Exception raised:** NONE

- **LUI:** Load Upper Immediate

**Format:** lui rd, imm[19:0]

**Description:** place the immediate value in the top 20 bits of the destination register rd, filling in the lowest 12 bits with zeros.

**Pseudocode:**  $x[rd] = sext(imm[31:12] \ll 12)$

**Invalid values:** NONE

**Exception raised:** NONE

- **AUIPC:** Add Upper Immediate to PC

**Format:** auipc rd, imm[19:0]

**Description:** form a 32-bit offset from the 20-bit immediate, filling in the lowest 12 bits with zeros, adds this offset to the pc, then place the result in register rd.

**Pseudocode:**  $x[rd] = pc + sext(immediate[31:12] \ll 12)$

**Invalid values:** NONE

**Exception raised:** NONE

## Integer Register-Register Instructions

- **ADD:** Addition

**Format:** add rd, rs1, rs2

**Description:** add rs2 to register rs1, and store the result in register rd.

**Pseudocode:**  $x[rd] = x[rs1] + x[rs2]$

**Invalid values:** NONE

**Exception raised:** NONE

- **SUB:** Subtraction

**Format:** sub rd, rs1, rs2

**Description:** subtract rs2 from register rs1, and store the result in register rd.

**Pseudocode:**  $x[rd] = x[rs1] - x[rs2]$

**Invalid values:** NONE

**Exception raised:** NONE

- **AND:** AND logical operator

**Format:** and rd, rs1, rs2

**Description:** perform bitwise AND on register rs1 and rs2 and place the result in rd.

**Pseudocode:**  $x[rd] = x[rs1] \& x[rs2]$

**Invalid values:** NONE

**Exception raised:** NONE

- **OR:** OR logical operator

**Format:** or rd, rs1, rs2

**Description:** perform bitwise OR on register rs1 and rs2 and place the result in rd.

**Pseudocode:**  $x[rd] = x[rs1] | x[rs2]$

**Invalid values:** NONE

**Exception raised:** NONE

- **XOR:** XOR logical operator



**Format:** xor rd, rs1, rs2

**Description:** perform bitwise XOR on register rs1 and rs2 and place the result in rd.

**Pseudocode:**  $x[rd] = x[rs1] \wedge x[rs2]$

**Invalid values:** NONE

**Exception raised:** NONE

- **SLT:** Set Less Than

**Format:** slt rd, rs1, rs2

**Description:** set register rd to 1 if register rs1 is less than rs2 when both are treated as signed numbers, else 0 is written to rd.

**Pseudocode:** if  $(x[rs1] < x[rs2])$   $x[rd] = 1$  else  $x[rd] = 0$

**Invalid values:** NONE

**Exception raised:** NONE

- **SLTU:** Set Less Than Unsigned

**Format:** sltu rd, rs1, rs2

**Description:** set register rd to 1 if register rs1 is less than rs2 when both are treated as unsigned numbers, else 0 is written to rd.

**Pseudocode:** if  $(x[rs1] <_u x[rs2])$   $x[rd] = 1$  else  $x[rd] = 0$

**Invalid values:** NONE

**Exception raised:** NONE

- **SLL:** Shift Left Logic

**Format:** sll rd, rs1, rs2

**Description:** logical left shift (zeros are shifted into the lower bits).

**Pseudocode:**  $x[rd] = x[rs1] \ll x[rs2]$

**Invalid values:** NONE

**Exception raised:** NONE

- **SRL:** Shift Right Logic

**Format:** srl rd, rs1, rs2

**Description:** logical right shift (zeros are shifted into the upper bits).

**Pseudocode:**  $x[rd] = x[rs1] \gg x[rs2]$

**Invalid values:** NONE

**Exception raised:** NONE

- **SRA:** Shift Right Arithmetic

**Format:** sra rd, rs1, rs2

**Description:** arithmetic right shift (the original sign bit is copied into the vacated upper bits).

**Pseudocode:**  $x[rd] = x[rs1] \ggg x[rs2]$

**Invalid values:** NONE

**Exception raised:** NONE

## Control Transfer Instructions

### Unconditional Jumps

- **JAL:** Jump and Link

**Format:** jal rd, imm[20:1]

**Description:** offset is sign-extended and added to the pc to form the jump target address (pc is calculated using signed arithmetic), then setting the least-significant bit of the result to zero, and store the address of instruction following the jump (pc+4) into register rd.

**Pseudocode:**  $x[rd] = pc+4$ ;  $pc += sext(imm[20:1])$

**Invalid values:** NONE

**Exception raised:** jumps to an unaligned address (4-byte or 2-byte boundary) will usually raise an exception.

- **JALR:** Jump and Link Register

**Format:** jalr rd, rs1, imm[11:0]

**Description:** target address is obtained by adding the 12-bit signed immediate to the register rs1 (pc is calculated using signed arithmetic), then setting the least-significant bit of the result to zero, and store the address of instruction following the jump (pc+4) into register rd.

**Pseudocode:**  $t = pc+4$ ;  $pc = (x[rs1] + sext(imm[11:0])) \& \sim 1$  ;  $x[rd] = t$

**Invalid values:** NONE

**Exception raised:** jumps to an unaligned address (4-byte or 2-byte boundary) will usually raise an exception.

## Conditional Branches

- **BEQ:** Branch Equal

**Format:** beq rs1, rs2, imm[12:1]

**Description:** takes the branch (pc is calculated using signed arithmetic) if registers rs1 and rs2 are equal.

**Pseudocode:** if  $(x[rs1] == x[rs2])$   $pc += sext(\{imm[12:1], 1'b0\})$  else  $pc += 4$

**Invalid values:** NONE

**Exception raised:** no instruction fetch misaligned exception is generated for a conditional branch that is not taken. An Instruction address misaligned exception is raised if the target address is not aligned on 4-byte or 2-byte boundary, because the core supports compressed instructions.

- **BNE:** Branch Not Equal



**Format:** bne rs1, rs2, imm[12:1]

**Description:** takes the branch (pc is calculated using signed arithmetic) if registers rs1 and rs2 are not equal.

**Pseudocode:** if ( $x[rs1] \neq x[rs2]$ )  $pc += sext(\{imm[12:1], 1'b0\})$  else  $pc += 4$

**Invalid values:** NONE

**Exception raised:** no instruction fetch misaligned exception is generated for a conditional branch that is not taken. An Instruction address misaligned exception is raised if the target address is not aligned on 4-byte or 2-byte boundary, because the core supports compressed instructions.

- **BLT:** Branch Less Than

**Format:** blt rs1, rs2, imm[12:1]

**Description:** takes the branch (pc is calculated using signed arithmetic) if registers rs1 less than rs2 (using signed comparison).

**Pseudocode:** if ( $x[rs1] < x[rs2]$ )  $pc += sext(\{imm[12:1], 1'b0\})$  else  $pc += 4$

**Invalid values:** NONE

**Exception raised:** no instruction fetch misaligned exception is generated for a conditional branch that is not taken. An Instruction address misaligned exception is raised if the target address is not aligned on 4-byte or 2-byte boundary, because the core supports compressed instructions.

- **BLTU:** Branch Less Than Unsigned

**Format:** bltu rs1, rs2, imm[12:1]

**Description:** takes the branch (pc is calculated using signed arithmetic) if registers rs1 less than rs2 (using unsigned comparison).

**Pseudocode:** if ( $x[rs1] <_u x[rs2]$ )  $pc += sext(\{imm[12:1], 1'b0\})$  else  $pc += 4$

**Invalid values:** NONE

**Exception raised:** no instruction fetch misaligned exception is generated for a conditional branch that is not taken. An Instruction address misaligned exception is raised if the target address is not aligned on 4-byte or 2-byte boundary, because the core supports compressed instructions.

- **BGE:** Branch Greater or Equal

**Format:** bge rs1, rs2, imm[12:1]

**Description:** takes the branch (pc is calculated using signed arithmetic) if registers rs1 is greater than or equal rs2 (using signed comparison).

**Pseudocode:** if ( $x[rs1] \geq x[rs2]$ )  $pc += sext(\{imm[12:1], 1'b0\})$  else  $pc += 4$

**Invalid values:** NONE

**Exception raised:** no instruction fetch misaligned exception is generated for a conditional branch that is not taken. An Instruction address misaligned exception is raised if the target address is not aligned on 4-byte or 2-byte boundary, because the core supports compressed instructions.

- **BGEU:** Branch Greater or Equal Unsigned

**Format:** bgeu rs1, rs2, imm[12:1]

**Description:** takes the branch (pc is calculated using signed arithmetic) if registers rs1 is greater than or equal rs2 (using unsigned comparison).

**Pseudocode:** if ( $x[rs1] \geq_u x[rs2]$ )  $pc += sext(\{imm[12:1], 1'b0\})$  else  $pc += 4$

**Invalid values:** NONE

**Exception raised:** no instruction fetch misaligned exception is generated for a conditional branch that is not taken. An Instruction address misaligned exception is raised if the target address is not aligned on 4-byte or 2-byte boundary, because the core supports compressed instructions.

## Load and Store Instructions

- **LB:** Load Byte

**Format:** lb rd, imm(rs1)

**Description:** loads a 8-bit value from memory, then sign-extends to 32-bit before storing in rd (rd is calculated using signed arithmetic), the effective address is obtained by adding register rs1 to the sign-extended 12-bit offset.

**Pseudocode:**  $x[rd] = sext(M[x[rs1] + sext(imm[11:0]))[7:0])$

**Invalid values:** NONE

**Exception raised:** loads with a destination of x0 must still raise any exceptions and action any other side effects even though the load value is discarded.

- **LH:** Load Halfword

**Format:** lh rd, imm(rs1)

**Description:** loads a 16-bit value from memory, then sign-extends to 32-bit before storing in rd (rd is calculated using signed arithmetic), the effective address is obtained by adding register rs1 to the sign-extended 12-bit offset.

**Pseudocode:**  $x[rd] = \text{sext}(M[x[rs1] + \text{sext}(\text{imm}[11:0])][15:0])$

**Invalid values:** NONE

**Exception raised:** loads with a destination of x0 must still raise any exceptions and action any other side effects even though the load value is discarded, also an exception is raised if the memory address isn't aligned (2-byte boundary).

- **LW:** Load Word

**Format:** lw rd, imm(rs1)

**Description:** loads a 32-bit value from memory, then storing in rd (rd is calculated using signed arithmetic). The effective address is obtained by adding register rs1 to the sign-extended 12-bit offset.

**Pseudocode:**  $x[rd] = \text{sext}(M[x[rs1] + \text{sext}(\text{imm}[11:0])][31:0])$

**Invalid values:** NONE

**Exception raised:** loads with a destination of x0 must still raise any exceptions and action any other side effects even though the load value is discarded, also an exception is raised if the memory address isn't aligned (4-byte boundary).

- **LBU:** Load Byte Unsigned

**Format:** lbu rd, imm(rs1)

**Description:** loads a 8-bit value from memory, then zero-extends to 32-bit before storing in rd (rd is calculated using unsigned arithmetic), the effective address is obtained by adding register rs1 to the sign-extended 12-bit offset.

**Pseudocode:**  $x[rd] = \text{zext}(M[x[rs1] + \text{sext}(\text{imm}[11:0])][7:0])$

**Invalid values:** NONE

**Exception raised:** loads with a destination of x0 must still raise any exceptions and action any other side effects even though the load value is discarded.

- **LHU:** Load Halfword Unsigned



**Format:** lhu rd, imm(rs1)

**Description:** loads a 16-bit value from memory, then zero-extends to 32-bit before storing in rd (rd is calculated using unsigned arithmetic), the effective address is obtained by adding register rs1 to the sign-extended 12-bit offset.

**Pseudocode:**  $x[rd] = \text{zext}(M[x[rs1] + \text{sext}(\text{imm}[11:0])][15:0])$

**Invalid values:** NONE

**Exception raised:** loads with a destination of x0 must still raise any exceptions and action any other side effects even though the load value is discarded, also an exception is raised if the memory address isn't aligned (2-byte boundary).

- **SB:** Store Byte

**Format:** sb rs2, imm(rs1)

**Description:** stores a 8-bit value from the low bits of register rs2 to memory, the effective address is obtained by adding register rs1 to the sign-extended 12-bit offset.

**Pseudocode:**  $M[x[rs1] + \text{sext}(\text{imm}[11:0])][7:0] = x[rs2][7:0]$

**Invalid values:** NONE

**Exception raised:** NONE

- **SH:** Store Halfword

**Format:** sh rs2, imm(rs1)

**Description:** stores a 16-bit value from the low bits of register rs2 to memory, the effective address is obtained by adding register rs1 to the sign-extended 12-bit offset.

**Pseudocode:**  $M[x[rs1] + \text{sext}(\text{imm}[11:0])][15:0] = x[rs2][15:0]$

**Invalid values:** NONE

**Exception raised:** an exception is raised if the memory address isn't aligned (2-byte boundary).

- **SW:** Store Word

**Format:** sw rs2, imm(rs1)

**Description:** stores a 32-bit value from register rs2 to memory, the effective address is obtained by adding register rs1 to the sign-extended 12-bit offset.

**Pseudocode:**  $M[x[rs1] + sext(imm[11:0])][31:0] = x[rs2][31:0]$

**Invalid values:** NONE

**Exception raised:** an exception is raised if the memory address isn't aligned (4-byte boundary).

## Memory Ordering

- **FENCE:** Fence Instruction

**Format:** fence pre, succ

**Description:** order device I/O and memory accesses as viewed by other RISC-V harts and external devices or coprocessors. Any combination of device input (I), device output (O), memory reads (R), and memory writes (W) may be ordered with respect to any combination of the same. Informally, no other RISC-V hart or external device can observe any operation in the successor set following a FENCE before any operation in the predecessor set preceding the FENCE, as the core support 1 hart, the fence instruction has no effect so we can considerate it as a nop instruction.

**Pseudocode:** No operation (nop)

**Invalid values:** NONE

**Exception raised:** NONE

## Environment Call and Breakpoints

- **ECALL:** Environment Call

**Format:** ecall

**Description:** make a request to the supporting execution environment, which is usually an operating system. The ABI for the system will define how parameters for the environment request are passed, but usually these will be in defined locations in the integer register file.

**Pseudocode:** RaiseException(EnvironmentCall)

**Invalid values:** NONE

**Exception raised:** Raise an Environment Call exception.

- **EBREAK:**Environment Break

**Format:** ebreak

**Description:** cause control to be transferred back to a debugging environment.

**Pseudocode:** RaiseException(Breakpoint)

**Invalid values:** NONE

**Exception raised:** Raise a Breakpoint exception.